PART B

CLASSES, PACKAGES ENCAPSULATION

YOUR LEARNING OBJECTIVES

- Classes expose behavior while protecting internal state.
- package and import resolve names; neither bypasses visibility.
- public, protected, package-private and private have distinct boundaries.
- Constructors, this, static and final control initialization and ownership.
- Composition makes dependencies explicit and keeps abstractions focused.

22 focused concept lessons • one infographic

Mini exercises and reasoned answers included

Organize Java source, enforce invariants and control cross-package access.

JAVA 17+ • Allman braces • Code font enlarged

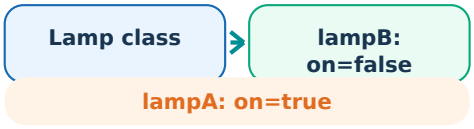
Anatomy of a class and its members

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A class groups instance state and operations. Fields describe what an object remembers; methods describe operations available through its type.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Lamp
{
    private boolean on;
    public void turnOn()
    {
        on = true;
    }
    public boolean isOn()
    {
        return on;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Class declaration creates the type Lamp; no Lamp instance exists yet.
2. When an instance is constructed, its on field starts false.
3. turnOn() changes that instance field; isOn() reads the same state.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Do not make fields public merely because the class is used widely. Methods let you preserve invariants while changing the representation later. For example, a balance operation should usually be deposit(amount) rather than setBalance(anything). Source readers should first identify the visible public API, then inspect private fields to understand how that API is implemented. A class declaration is a blueprint, not an object. Every new Lamp owns a distinct on field. Marking that field private means callers cannot set arbitrary values directly; they must use the supported methods. A method can read or change its receiver’s state even if other code cannot access that state.

COMMON TRAP

A class with only fields exposed publicly has very little encapsulation.

TRY THIS IN INTELLIJ

Create Lamp, call turnOn(), then print isOn().

REMEMBER turnOn() changes that instance field; isOn() reads the same state.

APPLY THE RULE / CHANGE ONE THING

Create two Lamps, turn on only the first, then call isOn() on both.

Answer: The first is true; the second remains false.

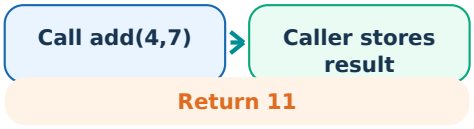
Methods, parameters, and return values

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A method declares a return type, name, parameters, and body. Calling it evaluates arguments, copies their values to parameters, and executes its statements.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static int add(int a, int b)
{
    return a + b;
}
int result = add(4, 7);
System.out.println(result);
```

EXPECTED OUTPUT

```
11
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The caller evaluates 4 and 7 before the call.
2. The parameters a and b receive copies of those int values.
3. return a + b evaluates to 11; the caller assigns it to result.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The parameter list is part of the method’s contract, and each argument expression is evaluated before the call. Returning does not implicitly alter caller locals; the caller must store the result if needed. A method may have side effects as well, which should be obvious from its name or documentation. Keep long methods focused on one coherent responsibility. The signature is a contract: return type, name, and ordered parameter types. The returned value goes to the call expression; it is not automatically displayed or retained. A void method produces no return value for assignment, whereas a method returning int may be used inside a larger expression.

COMMON TRAP

The method signature used for overloading excludes the return type; changing only the return type is not a valid overload.

TRY THIS IN INTELLIJ

Change return a+b to return a*b and predict the result.

REMEMBER return a + b evaluates to 11; the caller assigns it to result.

APPLY THE RULE / CHANGE ONE THING

Call add(4, 7) without storing or printing its result.

Answer: The method still returns 11, but nothing is printed.

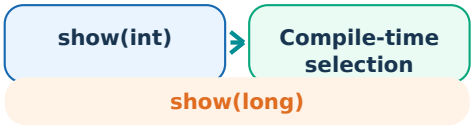
Overloading methods at compile time

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Methods can share a name if their parameter lists differ. Java chooses an applicable overload using compile-time types and conversion rules.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
static String show(int n)
{
    return "int";
}
static String show(long n)
{
    return "long";
}
System.out.println(show(4));
System.out.println(show(4L));
```

EXPECTED OUTPUT

```
int
long
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The compiler sees the literal 4 has type int.
2. The int overload is the closest applicable method; 4L has type long.
3. The selected methods return the strings int and long.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Java overload resolution considers method applicability and specificity, not just textual order. A literal 4 has type int, while 4L is long. Overloads with reference types can become ambiguous when passed null, and combinations of widening, boxing and varargs can surprise you. Avoid APIs that require readers to memorize a complicated overload hierarchy. Overloading is not about the return type alone. The compiler considers arity, parameter types, and allowed conversions. If two candidates are equally specific, it can reject a call as ambiguous. A change to the variable's declared type can change which overload is selected, even when the runtime numeric value stays the same.

COMMON TRAP

Overloading does not mean runtime polymorphism; ordinary overload selection is compile-time behavior.

TRY THIS IN INTELLIJ

Add show(double n). Which call matches 4.0?

REMEMBER The selected methods return the strings int and long.

APPLY THE RULE / CHANGE ONE THING

Add show(double) and call show(4.0).

Answer: The double overload is selected; integer literals still select show(int).

The package declaration identifies a namespace

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A package groups related types and controls certain access boundaries. Its declaration normally appears first, before imports and type declarations.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
package com.example.model;
public final class Person
{
    private final String name;
    public Person(String name)
    {
        this.name = name;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The package statement establishes the name com.example.model.
2. The public class Person belongs to that namespace regardless of who imports it.
3. The source path convention mirrors the dotted package path.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The package name appears in the fully qualified type name, such as com.example.model.Person. Files in different packages may declare the same simple class name. A package usually maps to a directory structure under the source root, but the Java language's package membership comes from declarations. Modern projects may add Java modules, which impose additional export rules. A package name is not an object or a folder that Java traverses at runtime. It is part of a type's fully qualified name and also defines package-private access. Build tools conventionally map the package to source directories so the compiler, IDE, and readers can locate code consistently.

COMMON TRAP

A folder called model by itself does not declare package model. The package statement is authoritative.

TRY THIS IN INTELLIJ

Move the file to a different directory but keep the package name. How might IntelliJ react?

REMEMBER The source path convention mirrors the dotted package path.

APPLY THE RULE / CHANGE ONE THING

Move Person.java to another folder without changing its package line.

Answer: The declared package remains com.example.model; an ordinary Maven layout becomes inconsistent.

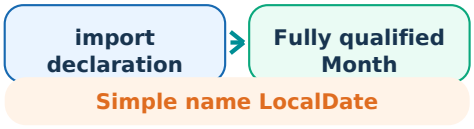
Imports and fully qualified names

Understand the rule • trace the code • read the diagram • test it

CORE RULE

An import lets unqualified simple class names refer to types in other packages. It does not copy or load the class and does not grant access. java.lang types are automatically available by simple name.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
import java.time.LocalDate;
LocalDate date = LocalDate.of(2026, 10, 10);
java.time.Month month = date.getMonth();
System.out.println(month);
```

EXPECTED OUTPUT

```
OCTOBER
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The import gives LocalDate a usable simple name.
2. The of method creates the chosen date value.
3. Month is referenced by its full name and prints OCTOBER.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Import is a source-level convenience. Importing a class has no effect on what methods are visible or whether a constructor is public. You can use a fully qualified name directly and remove the import. Static imports bring accessible static members into simple-name scope; they are useful for limited cases such as assertions or mathematical constants. Import statements affect name resolution during compilation. They do not instantiate classes, grant access to private types, or import dependencies into a build. A fully qualified name always works when that type is accessible. java.lang and classes in your current package do not require ordinary imports.

COMMON TRAP

Imports do not bypass public/package-private restrictions; an inaccessible class remains inaccessible.

TRY THIS IN INTELLIJ

Replace LocalDate with java.time.LocalDate everywhere.

REMEMBER Month is referenced by its full name and prints OCTOBER.

APPLY THE RULE / CHANGE ONE THING

Remove the LocalDate import and write java.time.LocalDate everywhere.

Answer: The result is unchanged; only source-name resolution differs.

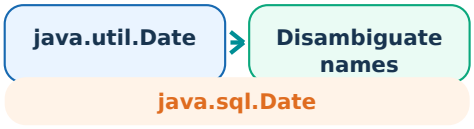
Wildcard imports and name collisions

Understand the rule • trace the code • read the diagram • test it

CORE RULE

import java.util.* makes eligible types in one package available by simple name; it does not recursively import subpackages. Importing both Date classes by their simple names would be ambiguous.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
import java.util.Date;
Date oldDate = new Date(0);
java.sql.Date sqlDate = new java.sql.Date(0);
System.out.println(oldDate.getTime());
System.out.println(sqlDate.getTime());
```

EXPECTED OUTPUT

```
0
0
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Import java.util.Date selects one Date name.
2. The SQL Date is explicitly qualified to avoid conflict.
3. Both objects represent the epoch start and return 0 milliseconds.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A wildcard import never means “all packages underneath this name.” When both java.sql.Date and java.util.Date are in scope, use a fully qualified name to make intent explicit. Ambiguous simple names are a compilation problem, not a runtime class-loading decision. Organized imports in IntelliJ do not change package boundaries. A wildcard import applies to accessible types in one named package only. It does not recursively include subpackages such as java.util.concurrent. Two on-demand imports that expose the same simple type name can force a fully qualified reference. Explicit naming is useful when legacy APIs define classes named Date, List, or Timer.

COMMON TRAP

java.util.* never imports java.util.concurrent.*; package hierarchies are naming conventions, not inheritance.

TRY THIS IN INTELLIJ

Try importing both java.util.Date and java.sql.Date and declaring Date x.

REMEMBER Both objects represent the epoch start and return 0 milliseconds.

APPLY THE RULE / CHANGE ONE THING

Replace the explicit Date import with both java.util.* and java.sql.*.

Answer: Unqualified Date becomes ambiguous; qualify or explicitly import the intended type.

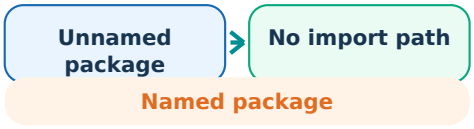
Why the default package causes trouble

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A class without a package declaration lives in the unnamed package. Named-package code cannot import types from that unnamed package.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
// Not recommended for a real project:
public class Scratch
{
    public static void main(String[] args)
    {
        System.out.println("scratch");
    }
}
```

EXPECTED OUTPUT

```
scratch
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Scratch has no package statement and enters the unnamed package.
2. Its main method prints scratch.
3. A class in a named package cannot import Scratch by simple name.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The unnamed package is convenient for a one-file toy program but interacts poorly with named packages and tooling. As soon as code contains multiple collaborating classes, choose a named package and align your source tree. Do not confuse a generated package folder in IntelliJ with the Java package declaration at the top of the file. Unnamed-package examples are acceptable for tiny scratch files, but they do not scale. Professional repositories use stable package names so tests, services, and libraries can reference types reliably. Simply moving a file into a folder does not create a package declaration automatically.

COMMON TRAP

Moving Scratch under a folder does not automatically make it a named-package class.

TRY THIS IN INTELLIJ

Give Scratch a package named com.example.app and align its folder.

REMEMBER A class in a named package cannot import Scratch by simple name.

APPLY THE RULE / CHANGE ONE THING

Put Scratch in the default package, then attempt import Scratch; from a named package.

Answer: The import is not legal; place Scratch in a named package.

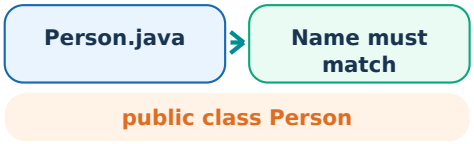
Public top-level classes and file names

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A public top-level Java class must normally be declared in a .java source file with the same simple name. A compilation unit may contain additional package-private top-level types, but only one public top-level type under the normal source-file conventions.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
// Person.java
package com.example.model;
public class Person
{
}
// Another separate file can declare
// package-private Helper in this package.
```

EXPECTED OUTPUT

```
No console output (compile-time rule).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Person is public and top-level.
2. The conventional source file is Person.java.
3. Other package-private top-level types may share the file, but only one public top-level type may be declared.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Multiple top-level package-private classes can appear in the same compilation unit, but one class per file is normally easier to maintain. The public class name must match the source filename used by ordinary Java compilation. A nested public class follows different naming rules because its enclosing top-level class owns the source file. The public type name and source file name must match case exactly in normal source compilation. A Java source file may contain multiple top-level declarations, but burying several major types together makes IDE navigation and maintenance harder. Public nested types follow a different rule because they are members of an enclosing type.

COMMON TRAP

Class and public filename matching is case-sensitive on Java source rules.

TRY THIS IN INTELLIJ

Name a file Human.java while it declares public class Person; observe the compiler diagnostic.

REMEMBER Other package-private top-level types may share the file, but only one public top-level type may be declared.

APPLY THE RULE / CHANGE ONE THING

Rename Person.java to People.java without renaming public class Person.

Answer: The compiler reports a public type/file-name mismatch.

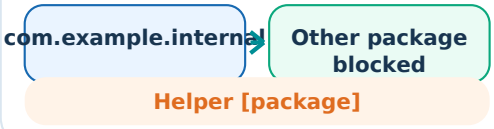
Package-private classes and helpers

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A top-level class without public is package-private. Other classes in the same package can use it, but different packages cannot.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
package com.example.internal;
class Helper
{
    String work()
    {
        return "internal";
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Helper lacks the public modifier.
2. Same-package classes can use Helper and its eligible members.
3. Code in unrelated packages cannot import this class as an accessible public API.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Package-private collaborators are useful for keeping an implementation behind a small public API. They can still be tested by tests located in the same package. However, package-private does not make a type secure from reflection or all possible runtime mechanisms. It is an access rule enforced by the language, not an encryption or sandbox boundary. Package-private is an intentional design boundary. It lets implementation classes collaborate without becoming part of a package's public contract. The restriction is checked by the compiler and language rules; it is not a security sandbox. Remember that com.example and com.example.internal are different packages.

COMMON TRAP

Importing com.example.internal.Helper from another package is not permitted just because its file exists.

TRY THIS IN INTELLIJ

Create a public InternalFacade that delegates to Helper.

REMEMBER Code in unrelated packages cannot import this class as an accessible public API.

APPLY THE RULE / CHANGE ONE THING

Create com.example.app.Main and try new Helper() from it.

Answer: Access is rejected; expose a public facade or change API design.

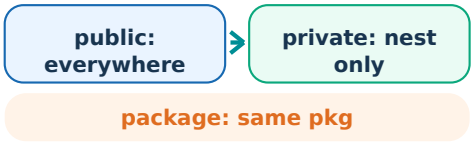
The four member access levels

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Access control limits where code can use classes and members. Public is broadly visible; private is within the enclosing top-level nest; package-private is same package. protected adds an inheritance-based path for code in other packages, with receiver restrictions.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
public class Vault
{
    public int id;
    protected int level;
    int internalCode; // package-private
    private String password;
}
```

EXPECTED OUTPUT

```
No console output (declaration).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Each member uses a different modifier.
2. Visibility depends on caller package and inheritance context.
3. private is restricted to enclosing nest; public is broadly visible.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

The access matrix is easy to memorize incorrectly. public does not bypass module exports or all runtime access controls. private applies inside the enclosing nest, including related nested types under modern Java nest-based access. package-private access depends on the actual runtime package identity, including loader/module subtleties beyond this foundation. There are only two kinds of access for top-level classes: public and package-private. All four modifiers apply to ordinary class members. protected is the subtle one: it includes same-package access plus a restricted path for subclasses outside the package. An import never changes this matrix.

COMMON TRAP

The default access level is not protected. An omitted modifier means package-private.

TRY THIS IN INTELLIJ

Draw the same-class/same-package/subclass/unrelated access matrix.

REMEMBER private is restricted to enclosing nest; public is broadly visible.

APPLY THE RULE / CHANGE ONE THING

Try to read password and internalCode from a class in a different package.

Answer: Both are inaccessible (unless the code is in a permitted enclosing nest for private).

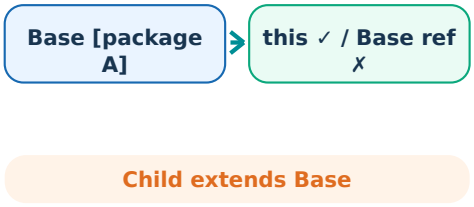
protected across package boundaries

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A subclass in a different package can use an inherited protected member through an appropriate subclass reference (such as this), not any base instance. Cross-package protected access is deliberately narrower than “subclasses can access protected fields on any object.” The companion project contains AccessBase in access.base and AccessChild in access.child so you can test the compiler boundary.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
// In package child:
class Child extends Base
{
    String show()
    {
        return this.protectedValue;
    }
    // new Base().protectedValue is not allowed.
}
```

EXPECTED OUTPUT

```
No console output (access example).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. A child class extends the base from another package.
2. Within the subclass, this.protectedValue is inherited and permitted.
3. new Base().protectedValue is not permitted in that cross-package context.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Across packages, protected access requires a subclass context and an appropriate receiver of that subclass type (or its subtype). Inside the same package, the inheritance receiver limitation does not apply in the same way. Verify protected behavior by compiling the two-package sample, rather than relying on a one-line yes/no table. In a different package, protected access is not unrestricted access to all superclass instances. From instance code of a subclass S, an appropriate qualifying expression must have type S or a subtype of S. This prevents a subclass from reaching protected members on arbitrary base-class objects created elsewhere.

COMMON TRAP

Do not infer protected access from subclass status alone; inspect the receiver expression type.

TRY THIS IN INTELLIJ

Try accessing protectedValue through a new AccessBase() inside AccessChild.

REMEMBER new Base().protectedValue is not permitted in that cross-package context.

APPLY THE RULE / CHANGE ONE THING

In Child, declare Base other = new Base(); and access other.protectedValue.

Answer: The access fails; use an appropriate Child receiver or accessible base API.

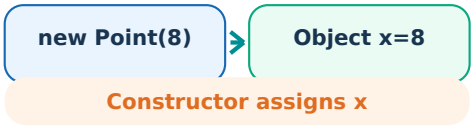
Constructors establish initial object state

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A constructor initializes a new object. Its name matches the class, and it has no return type, not even void.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Point
{
    private final int x;
    Point(int x)
    {
        this.x = x;
    }
    int getX()
    {
        return x;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. new Point(8) allocates a Point instance.
2. The constructor receives x=8 and assigns this.x once.
3. The getter would later return that object’s field value, 8.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A constructor cannot declare a return type. When a class declares any constructor, the compiler does not synthesize another no-argument constructor. Constructors can delegate to each other, validate inputs and establish final fields. Avoid performing overridable calls, starting threads, or leaking references before initialization is complete. Constructors establish invariants as objects become usable. Before their bodies execute, fields have default values; field initializers and superclass initialization also participate. A constructor has no declared return type. If you write void Point(), you have created an ordinary method, not a constructor.

COMMON TRAP

A method named Point with a void return type is not a constructor.

TRY THIS IN INTELLIJ

Try new Point() and explain the compile error.

REMEMBER The getter would later return that object’s field value, 8.

APPLY THE RULE / CHANGE ONE THING

Add an argument check that rejects x below zero.

Answer: The constructor can throw before a reference to a valid Point is returned.

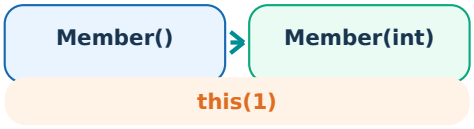
Constructor overloading and chaining with this

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Multiple constructors can offer alternative ways to create a valid object. this(...) calls another constructor of the same class. Explicit constructor invocation with this(...) must be first in a constructor body in Java 17, and chaining cannot be cyclic.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Member
{
    private final int level;
    Member()
    {
        this(1);
    }
    Member(int level)
    {
        this.level = level;
    }
}
```

EXPECTED OUTPUT

```
No console output (constructor definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Member() is selected for the no-argument call.
2. Its first action delegates to Member(1) via this(1).
3. The delegated constructor assigns level exactly once.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Constructor chaining is most useful when one constructor is the single validation path. In Java 17, this(...) or super(...) must be the first constructor statement, and you cannot form cycles. If a child superclass has no accessible no-argument constructor, the child must explicitly select an appropriate super(...). This is a frequent error when changing a base class API. Constructor chaining centralizes validation and assignments, reducing inconsistencies between overloads. For Java 17 source, an explicit this(...) or super(...) constructor invocation must be the first statement; it is not an ordinary method call and cannot recur in a cycle. A constructor eventually invokes a superclass constructor.

COMMON TRAP

Do not write both this(...) and super(...) in one constructor; one explicit constructor invocation occupies that first position.

TRY THIS IN INTELLIJ

Add Member(int level, String name) and chain to Member(int).

REMEMBER The delegated constructor assigns level exactly once.

APPLY THE RULE / CHANGE ONE THING

Change Member() to call this(3) instead.

Answer: New Member() then stores level=3.

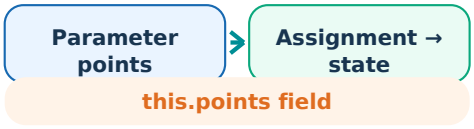
this: current receiver and shadowed names

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Inside an instance method or constructor, this refers to the current receiver object. It distinguishes fields from same-named parameters.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Score
{
    private int points;
    void setPoints(int points)
    {
        this.points = points;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. setPoints receives the parameter named points.
2. this.points refers to the field on the current Score object.
3. The parameter value is copied into the field.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

this can select fields, invoke another constructor with this(...), or be used as the receiver for an instance method. A parameter with the same name as a field shadows it; this.field restores clarity. There is no this inside a static context because no instance receiver exists there. Keep this escape out of constructors when practical. The receiver is the object whose instance method is being executed. this is not available in a static method because static methods do not have an implicit receiver. Writing this.field is especially useful when fields and parameters have the same name; it can also improve readability when several objects of the same type interact.

COMMON TRAP

this is unavailable in a static method because static methods have no receiver object.

TRY THIS IN INTELLIJ

Remove this. Predict whether the object's points field changes.

REMEMBER The parameter value is copied into the field.

APPLY THE RULE / CHANGE ONE THING

Remove this. and leave points = points;

Answer: Only the parameter assigns to itself; the field does not change.

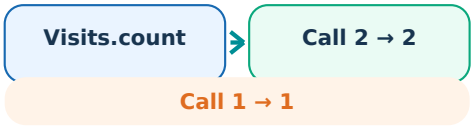
static state belongs to the class

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A static field has one shared value for a class, not a separate field for every instance. Static methods can be called without an instance.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Visits
{
    private static int count;
    static int next()
    {
        return ++count;
    }
}
System.out.println(Visits.next());
System.out.println(Visits.next());
```

EXPECTED OUTPUT

```
1
2
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Visits.count belongs to the class, not individual objects.
2. The first next() increments the shared count from 0 to 1.
3. The second call increments that same field to 2.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A static field is shared class-wide and may survive across many object creations. This is useful for constants, metadata and carefully designed registries. Mutable static state complicates deterministic tests and concurrent execution. Prefer dependencies passed explicitly rather than a globally changeable service reachable through a static field. Static members are associated with the class and are usually accessed through the class name. Shared mutable static state needs special care when multiple threads are present: ++count is not an atomic multi-threaded increment. Constants and stateless utility methods are safer typical uses of static.

COMMON TRAP

A mutable static counter is shared state and requires synchronization if multiple threads update it.

TRY THIS IN INTELLIJ

How do two separate Visits objects affect the one shared count?

REMEMBER The second call increments that same field to 2.

APPLY THE RULE / CHANGE ONE THING

Create two Visits instances and call next() through either one.

Answer: The single static count keeps increasing; instance creation does not reset it.

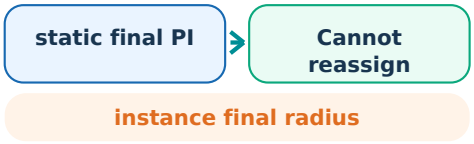
final fields, constants, and immutability

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A blank final instance field must be definitely assigned during object initialization. A static final constant belongs to the class. final references cannot be rebound, but a final field containing a mutable object does not automatically make that object immutable.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Circle
{
    static final double PI = 3.14159;
    private final double radius;
    Circle(double radius)
    {
        this.radius = radius;
    }
}
```

EXPECTED OUTPUT

```
No console output (class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The class-level PI constant is initialized once.
2. Each Circle instance receives its own final radius.
3. Each constructor must assign that blank final field before completion.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A final field must be definitely assigned once along all constructor paths. A blank final field may be assigned in an initializer or constructor as permitted. Static final compile-time constants may be inlined into clients, so changing a public constant without rebuilding dependent code can surprise you. Final fields support, but do not guarantee, a wholly immutable object design. final prevents reassignment of a field after definite initialization. It does not mean an object graph is deeply immutable; a final List field can still refer to a mutable list. Truly immutable classes also avoid leaking mutable references and are designed so their valid state is fixed after construction.

COMMON TRAP

Avoid public static final mutable objects as globally editable pseudo-constants.

TRY THIS IN INTELLIJ

What changes if radius is initialized where it is declared rather than in the constructor?

REMEMBER Each constructor must assign that blank final field before completion.

APPLY THE RULE / CHANGE ONE THING

Change radius to final List<String> names; then add to the list.

Answer: The reference cannot be reassigned, but the list itself can still be mutated.

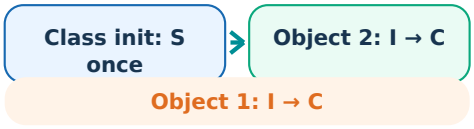
Initialization order: static, instance, constructor

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Static initialization normally occurs once when the class is initialized. Creating each object executes instance initialization before the constructor body.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Order
{
    static
    {
        System.out.print("S");
    }
    {
        System.out.print("I");
    }
    Order()
    {
        System.out.print("C");
    }
}
new Order();
new Order();
```

EXPECTED OUTPUT

```
SICIC
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The static initializer prints S once when Order initializes.
2. Each new Order executes the instance initializer, printing I.
3. Each constructor then prints C; two instances yield SICIC.

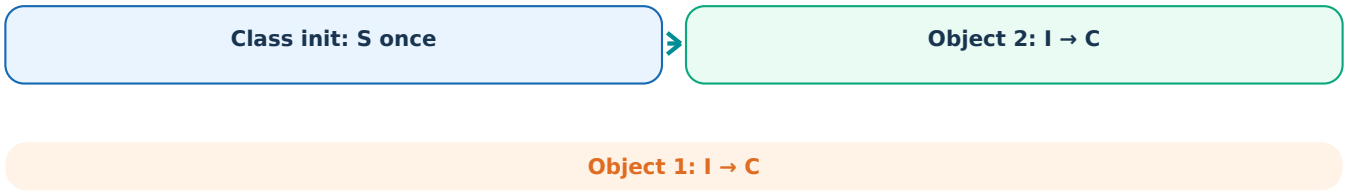
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Initialization order: static, instance, constructor

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

The static initializer prints S once when Order initializes. Each new Order executes the instance initializer, printing I. Each constructor then prints C; two instances yield SICIC.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Initialization order is easier to reason about in phases: class initialization, object allocation/defaults, superclass initialization, then subclass field initializers and constructor logic. Inheritance adds another layer: subclass overridable methods can see default values before their own initialization. When behavior depends on order, prefer simple constructors and explicit dependency creation. Initialization includes default field values, explicit initializers, superclass construction, and constructor bodies. The exact sequence gets more complex under inheritance; first master the once-per-class versus once-per-object distinction. A static initializer normally runs at first active use, not necessarily when a class file is first loaded.

COMMON TRAP

Do not confuse class loading, linking, and initialization; the simple mental model is not the whole JVM specification.

TRY THIS IN INTELLIJ

Predict output if only one Order object is constructed.

REMEMBER Each constructor then prints C; two instances yield SICIC.

APPLY THE RULE / CHANGE ONE THING

Create a third Order.

Answer: The third instance adds IC, producing SICICIC.

Encapsulation means preserving invariants

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Encapsulation is not just adding getters and setters. The class must prevent invalid states and expose operations with meaningful guarantees.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
public void setAge(int age)
{
    if (age < 0 || age > 130)
    {
        throw new IllegalArgumentException("age");
    }
    this.age = age;
}
```

EXPECTED OUTPUT

```
No console output for valid setter calls.
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The setter receives an incoming age.
2. Invalid values cause an exception before state changes.
3. Only acceptable ages reach the assignment to this.age.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

An invariant is a property that should be true whenever a constructed object is available for normal use. For Person, age cannot be negative and name cannot be blank. Validate both the constructor path and any state-changing methods; otherwise a setter may undo constructor validation. For some concepts, no public setter is the clearest design. Encapsulation protects meaningful invariants, not simply field visibility. A public setter for every private field may still permit inconsistent state changes across multiple fields. Prefer operations expressing domain intent (deposit, approve, close) when a single unconstrained setter would permit invalid transitions.

COMMON TRAP

A setter that accepts every value can defeat the very invariants encapsulation should protect.

TRY THIS IN INTELLIJ

Decide whether id should have a public setter.

REMEMBER Only acceptable ages reach the assignment to this.age.

APPLY THE RULE / CHANGE ONE THING

Call setAge(-1) after an object already has age 30.

Answer: An exception is thrown and the earlier valid age remains unchanged.

Defensive copying when returning mutable state

Understand the rule • trace the code • read the diagram • test it

CORE RULE

A getter that returns a direct reference to mutable internal state may allow callers to change that state without validation. A Date copy prevents the caller from mutating the stored Date through the getter.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
private Date created = new Date();
public Date getCreated()
{
    return new Date(created.getTime());
}
```

EXPECTED OUTPUT

```
No console output (getter definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The getter reads the internal Date timestamp.
2. It constructs and returns a distinct Date object.
3. The caller can mutate the returned Date without changing the original field.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A getter returning a mutable Date reference exposes an internal escape route. The caller may change it without calling a validated setter. Defensive copying is useful for Date, arrays and mutable collections, while immutable LocalDate and immutable Address can often be returned directly. The right choice depends on whether the field type itself can change after creation. Defensive copying is usually needed in both directions: copy mutable constructor inputs, and copy mutable outputs. A shallow copy can still share nested mutable components. New Java code should often prefer immutable java.time types, reducing defensive-copy complexity.

COMMON TRAP

A getter need not expose a field directly at all; exposing mutable implementation types couples clients to internals.

TRY THIS IN INTELLIJ

Change return to return created; explain the aliasing leak.

REMEMBER The caller can mutate the returned Date without changing the original field.

APPLY THE RULE / CHANGE ONE THING

Replace return new Date(created.getTime()) with return created.

Answer: A caller could then mutate internal state through the returned reference.

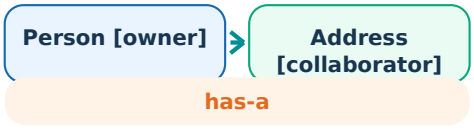
Composition: has-a relationships

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Composition means one object holds references to collaborators or value objects and delegates work to them. It is distinct from inheritance.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Person
{
    private final Address address;
    Person(Address address)
    {
        this.address = address;
    }
    String city()
    {
        return address.getCity();
    }
}
```

EXPECTED OUTPUT

```
No console output (relationship definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The Person constructor receives an Address reference.
2. The Person stores that collaborator as an instance field.
3. city() delegates to address.getCity() rather than inheriting Address.

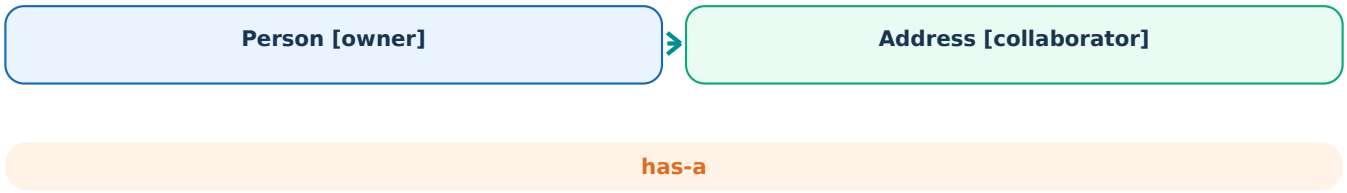
NEXT PAGE / EXPAND THE REASONING

Continue to the worked explanation, common trap, and Apply the Rule variation on the next page.

Composition: has-a relationships

Understand the rule • trace the code • read the diagram • test it

VISUAL MODEL



RECAP / HOW THE EXAMPLE BEHAVES

The Person constructor receives an Address reference. The Person stores that collaborator as an instance field. city() delegates to address.getCity() rather than inheriting Address.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

“Has-a” by itself does not define a lifetime relationship. Composition, aggregation and dependency are design-level distinctions that matter when deciding who creates, owns and replaces an object. Constructor injection makes required collaborators explicit; a method parameter may be better for a one-time helper. Use inheritance only when substitutability, not convenience, justifies the subtype. Composition expresses has-a relationships and enables swapping collaborations at construction time. A final Address field does not necessarily make the Address itself immutable; ownership still matters. Composition commonly creates smaller classes with clearer responsibilities and is the default choice when no genuine substitutable is-a relationship exists.

COMMON TRAP

Not every field relationship implies strict lifecycle ownership; “has-a” is a broad design shorthand.

TRY THIS IN INTELLIJ

Replace Address with a service interface and explain how it supports swapping implementations.

REMEMBER city() delegates to address.getCity() rather than inheriting Address.

APPLY THE RULE / CHANGE ONE THING

Supply a different Address implementation or object at construction.

Answer: The Person can reuse the same city() behavior without inheritance.

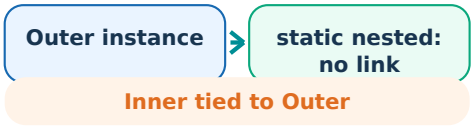
Nested, static nested, and inner classes

Understand the rule • trace the code • read the diagram • test it

CORE RULE

Java supports types declared inside other types. A static nested class has no implicit outer instance; a non-static inner class is tied to an outer instance.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
class Outer
{
    private int number = 5;
    class Inner
    {
        int read()
        {
            return number;
        }
    }
}
```

EXPECTED OUTPUT

```
No console output (nested class definition).
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. Outer has a per-instance number field.
2. Its non-static Inner type can read Outer.this.number implicitly.
3. A static nested class would not have that implicit outer object.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

A static nested class is often a good fit for a helper that logically belongs to an outer class but needs no outer instance. A non-static inner class stores a relationship to an enclosing instance and can access its members. Nested types have their own access rules; this becomes important when reading builder classes and internal implementation helpers. A non-static inner instance carries an association to an enclosing instance, which can keep that outer object reachable. A static nested class is often preferable when the nested type does not need instance data. A local class is declared inside a method; an anonymous class is an expression creating a one-off subtype/implementation.

COMMON TRAP

Static nested classes are not “top-level static classes”; they remain members of their enclosing type.

TRY THIS IN INTELLIJ

Convert Inner to static and observe why direct access to number no longer works.

REMEMBER A static nested class would not have that implicit outer object.

APPLY THE RULE / CHANGE ONE THING

Make Inner static without changing read().

Answer: The reference to Outer’s instance field then fails until an Outer is provided.

Read the companion project’s package graph

Understand the rule • trace the code • read the diagram • test it

CORE RULE

The IntelliJ project demonstrates a conventional app/model/service/internal/access hierarchy. Package names express namespaces; dependencies express actual relationships.

VISUAL MODEL



CODE EXAMPLE / ALLMAN BRACES

```
import com.refresher.ch3.model.Person;
import com.refresher.ch3.service.PersonService;
PersonService service = new PersonService(
    new FriendlyGreetingService());
System.out.println(service.welcome(person));
```

EXPECTED OUTPUT

```
Hello, Asha Rao! (with project demo values)
```

EXECUTION TRACE / WHAT JAVA ACTUALLY DOES

1. The app package contains the entry point and assembly code.
2. The model package contains state-holding types.
3. The service and implementation packages define contracts and collaborators.

DEEPER UNDERSTANDING / WHY THE DISTINCTION MATTERS

Read a multi-package project from the app entry point outward: identify imported public interfaces and classes, follow constructor arguments, then trace calls to implementations. Use Ctrl+B in IntelliJ to jump to declarations and implementations. Remember that folder names, imports, package declarations and visibility are four different concepts, even when they appear together in the source tree. Package graphs help reveal dependency direction. When you open an unfamiliar repository, navigate from the entry point into services and then their dependencies, rather than reading every class sequentially. A package tree is an organization convention, not an automatic dependency or security policy.

COMMON TRAP

Parent and child package names do not imply greater visibility or inheritance between packages.

TRY THIS IN INTELLIJ

Open PersonService.java, navigate to GreetingService, then follow the implementation.

REMEMBER The service and implementation packages define contracts and collaborators.

APPLY THE RULE / CHANGE ONE THING

Trace MainApp → PersonService → GreetingService implementation.

Answer: You see constructor injection of an interface-based collaborator.

CHAPTER 3 · PART B · VISUAL MASTER MAP

Core rules • mental models • common pitfalls • executable examples

01 CLASS STRUCTURE

- Fields hold state; methods define behavior.
- Constructors establish valid objects.
- Keep data private, expose small APIs.

02 PACKAGES & IMPORTS

- package identifies a type namespace.
- Imports shorten type names only.
- Subpackages are separate packages.

03 ACCESS CONTROL

- public: any accessible caller.
- private: enclosing nest.
- package-private: same package only.



04 PROTECTED RULE

- Same package may access it.
- Other-package subclasses have receiver restrictions.
- Avoid exposing fields unnecessarily.

05 INITIALIZATION

- this distinguishes current receiver.
- static is class-wide state.
- final fields must be assigned once.

06 COMPOSITION

- Has-a collaborators via fields.
- Validate invariants in constructors/methods.
- Prefer injection of abstractions.

EXAMPLE — SOURCE AND OUTPUT

CODE / RULE

```
package com.example.model;
public final class Person
{
    private final int id;
}
```

EXPECTED RESULT

```
No console output: type definition
```

PART B / PRACTICE

Predict before running

9 thinking questions. Answer from the Java rule, not from memorized output.

B1 Does import allow access to a package-private class in another package?

B2 Does package com.a.child count as the same package as com.a?

B3 What is the difference between protected and package-private?

B4 Can an unrelated class in a different package read protected?

B5 Why does this.field distinguish a parameter from a field?

B6 What happens to an implicit no-arg constructor after declaring one constructor?

B7 What restrictions apply to this(...) inside a Java 17 constructor?

B8 What does static say about a field?

B9 What makes a getter unsafe for mutable state?

PART B / ANSWER KEY

Reasoning and answers

Explain each rule in your own words before moving on.

B1 No. Import resolution does not change type accessibility.

B2 No. Package names do not confer hierarchical access.

B3 Package-private allows only same-package access; protected also offers restricted subclass access across packages.

B4 No, not through ordinary access merely because the member exists.

B5 this identifies the current receiver object and selects its field.

B6 The compiler stops providing a default no-arg constructor automatically.

B7 It must appear first and chaining may not be cyclic.

B8 There is one class-level field rather than one field per instance.

B9 Returning a direct mutable internal reference lets callers bypass validation or invariants.